

WEEK 1 — ADVANCED PYTHON FOR AI

DAY 1 — FUNCTIONS DEEPLY

1. **What is Function:** A function is a reusable block of logic that takes some input and optionally produces an output.

```
def add(a, b): #defining a function
    return a + b
result = add(5, 3) #calling the function
print(result)
```

2. Parameters Vs Arguments:

Parameter → variable defined in function (a,b)

Argument → actual value passed to function (5,3) :

Types of Arguments:

- a. Positional Arguments: python matches the arguments and parameter according to the position if name(parameter) 1st position par he then hame (argument) bhi us position par hi dena hoga

```
def introduce(name, age):
    print(name, age)
introduce("Aastha", 21)
```

- b. Keyword Argument: `introduce(age=21, name="Aastha")` here we have changes the position of the parameters according we will be giving the arguments ok this is b=very important when we have the multiple parameters.

Default Argument: if no argument is given it will take the argument jo parameter ke sath likha he else ham jo message ka argument denge vo aa jaenga.

```
def greet(name, message="Hello"):
    print(message, name)
greet("Aastha")
```

- c. Variable-length positional arguments — `*args`: the function that can accept any number of arguments that is called as variable length positional arguments, here args is a tuple

```
def add(*args): #
    print(args)
add(1, 2, 3, 4, 5) #output: (1, 2, 3, 4, 5)
def add(*args):
    total = 0
    for x in args: #
        total += x

    return total
```

- d. Variable-length keyword arguments — ****kwargs**: here the kwargs is in the form of dictionary, iske liye hame keywords define karne hote he.

```
def information(**kwargs):
    print(kwargs)
information(name="Aastha", age=21, branch="CSE")
#output:
{
    'name': 'Aastha',
    'age': 21,
    'branch': 'CSE'
}
```

1. **why does AI engineering care?** So for this answer consider a ML model configuration in that you might have

```
def train_model(model, X, y, **config):
    ...
# then
train_model(
    model,
    X,
    y,
    learning_rate=0.001,
    epochs=20,
    batch_size=32
)
#then config will be captured in the way:
{
    "learning_rate": 0.001,
    "epochs": 20,
    "batch_size": 32
}
```

2. **Functions are first-class objects**: this means in python the functions are treated as an object like for an example:

```
def square(x):
    return x * x
#we can do:
f = square #here f means the square function, 1. here we are storing the function
in the variable f
print(f(5)) #output will be 25
2. Function ko kisi doosre function mein argument ke roop mein pass karna:
def square(x):
    return x * x
def apply_operation(x, operation):
    return operation(x)
```

```
result = apply_operation(5, square)
print(result)
```

3. Function ko function se return karna:

```
def create_multiplier(n):
    def multiply(x):
        return x * n
    return multiply
double = create_multiplier(2)
print(double(5))
```

4. Functions ko list/dictionary mein store karna:

```
def square(x):
    return x * x
def cube(x):
    return x * x * x
operations = [square, cube]    #list
print(operations[0](3)) #9
print(operations[1](3)) #27

operations = {                  #dictionary
    "square": square,
    "cube": cube
}
print(operations["square"](4)) #16
```

3. **Scope — Local, Global, Nonlocal:** variable ko function ya program ke kis part mein access kar sakte he that is called as the Scope.
- a. Local Variable: function ke ander banaye gaye variables jo sirf function mein hi hote he like for an example:

```
def test():
    x = 10
    print(x)
test()
print(x)    #error because function ke under wala variable bahar se access
            #nahi ho sakta
```

- b. Global Variable: function ke bahar wala variable jo hum define karte he jo ki we also can use inside the function called as global.

```
x = 10
def test():
    x = 20
    print(x)
test()
print(x)
```

- c. Nonlocal Variable: if inner function ko outer function ke variable ko modify karna he then we use this non local variable. Like for an example:

```
def outer():
    x = 10
    def inner():
        nonlocal x
        x = 20
    inner()
    print(x)
outer()
```

1. Scope hierarchy: LEGB

- Local
- Enclosing
- Global
- Built-in

2. Mutable vs Immutable Arguments:

- Mutable: int, float, str, tuple, bool
- Immutabale: list, dict, set

3. Common Function Traps:

- Trap 1 — Mutable default argument:

Isme Default list [] function definition ke time ek baar create hoti hai, har function call par nahi. So same list reuse ho rahi hai. Like for an example:

```
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
print(add_item("A")) # output: ["A"]
print(add_item("B")) #output: ["A","B"]
```

- Trap 2 — return vs print:

`print()` → screen par output

`return` → function se value bahar bhejna

- Trap 3 — Function call vs function itself:

Let's assume the function that is:

```
def square(x):
    return x * x
square          #this is function itself
square(5)       #but this is function call
apply_operation(5, square) # here we have passed the function not its result
```

DAY 2 — Python Data Structures for AI

1. The 4 Core Data Structures:

Structure	Ordered?	Mutable?	Duplicates?	Access
List []	✓	✓	✓	index
Tuple ()	✓	✗	✓	index
Set set()	✗*	✓	✗	membership
dict {}	✓**	✓	keys unique	key

2. Why dictionaries are extremely important in AI:

We will see the dictionary everywhere in the in ai for example:

This becomes especially important later in RAG, APIs, model serving, and agent systems.

- Model configuration:

```
config = {
    "learning_rate": 0.001,
    "batch_size": 32,
    "epochs": 20
}
```
- Prediction output:

```
prediction = {
    "label": "cat",
    "confidence": 0.94
}
```

3. Time Complexity:

- LIST:

Operation	Average complexity
<code>arr[i]</code>	$O(1)$
<code>append()</code>	$O(1)$ amortized
search <code>x in arr</code>	$O(n)$
insert at beginning	$O(n)$
delete from beginning	$O(n)$

- SET():

Operation	Average
<code>x in set</code>	O(1)
<code>add()</code>	O(1)
<code>remove()</code>	O(1)

- Dictionary:

Operation	Average
<code>d[key]</code>	O(1)
insert	O(1)
update	O(1)
delete	O(1)
key lookup	O(1)

4. Nested Data Structures:

In ai applications we will frequently see this:

```
users = [
    {
        "name": "Aastha",
        "skills": ["Python", "ML"]
    },
    {
        "name": "Rahul",
        "skills": ["Java", "SQL"]
    }
]
print(users[0]["skills"][0]) #output: Python
```

1.

```
names = ["A", "B", "C"]
marks = [80, 90, 85]
result = dict(zip(names, marks))
print(result)                # {'A': 80, 'B': 90, 'C': 85}
```

2.

```
list = [
    {"name": "A", "marks": [80, 90, 85]},
    {"name": "B", "marks": [70, 75, 80]},
    {"name": "C", "marks": [90, 95, 92]}
]
for i in list:
    print(i["name"], max(i["marks"]))
```